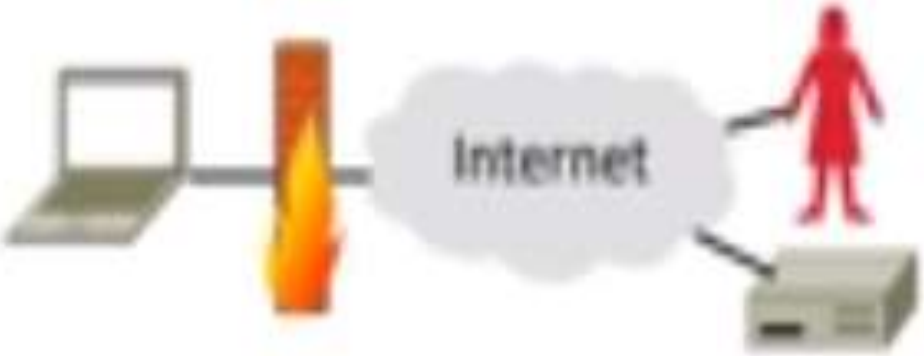
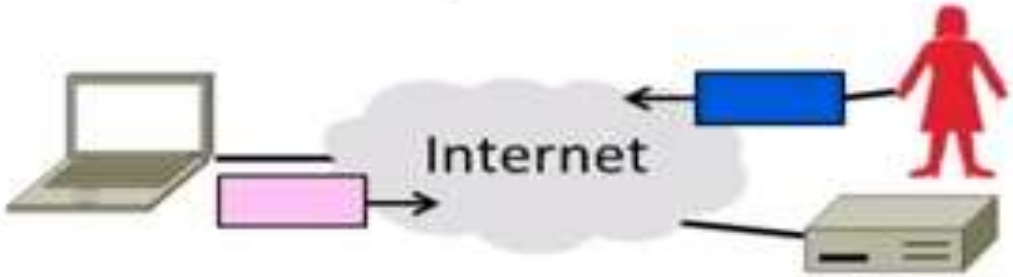
Topic

- Firewalls
  - Protecting hosts by restricting network connectivity



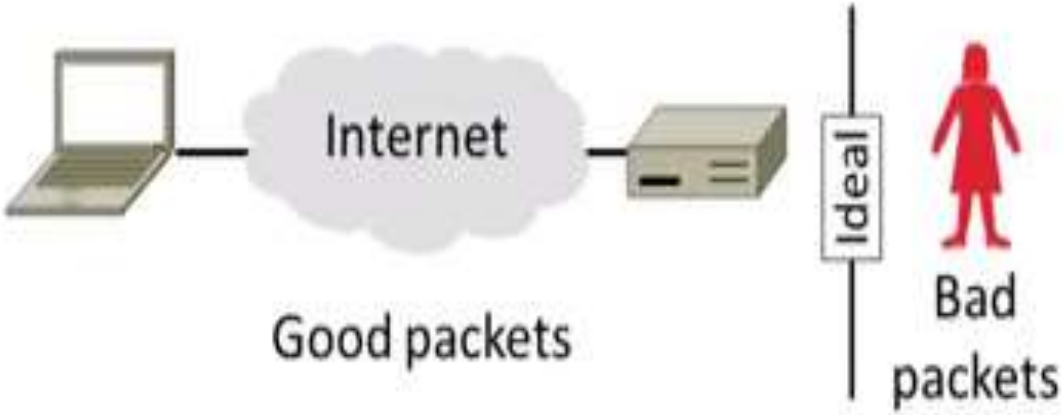
Motivation

- The best part of IP connectivity
  - You can send to any other host
- The worst part of IP connectivity
  - Any host can send packets to you!
  - There's nasty stuff out there ...



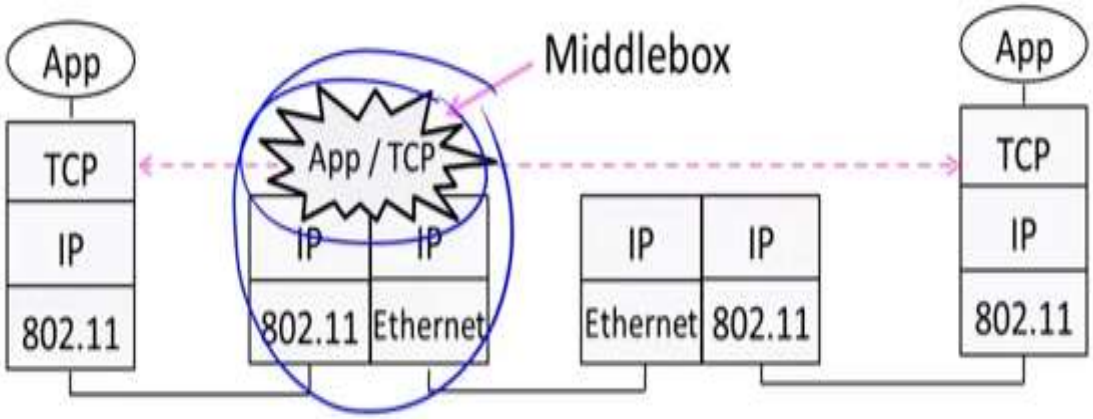
Goal and Threat Model

- Goal of firewall is to implement a boundary to restrict IP connectivity:
  - You can talk to hosts as intended
  - Trudy can't talk to you over network

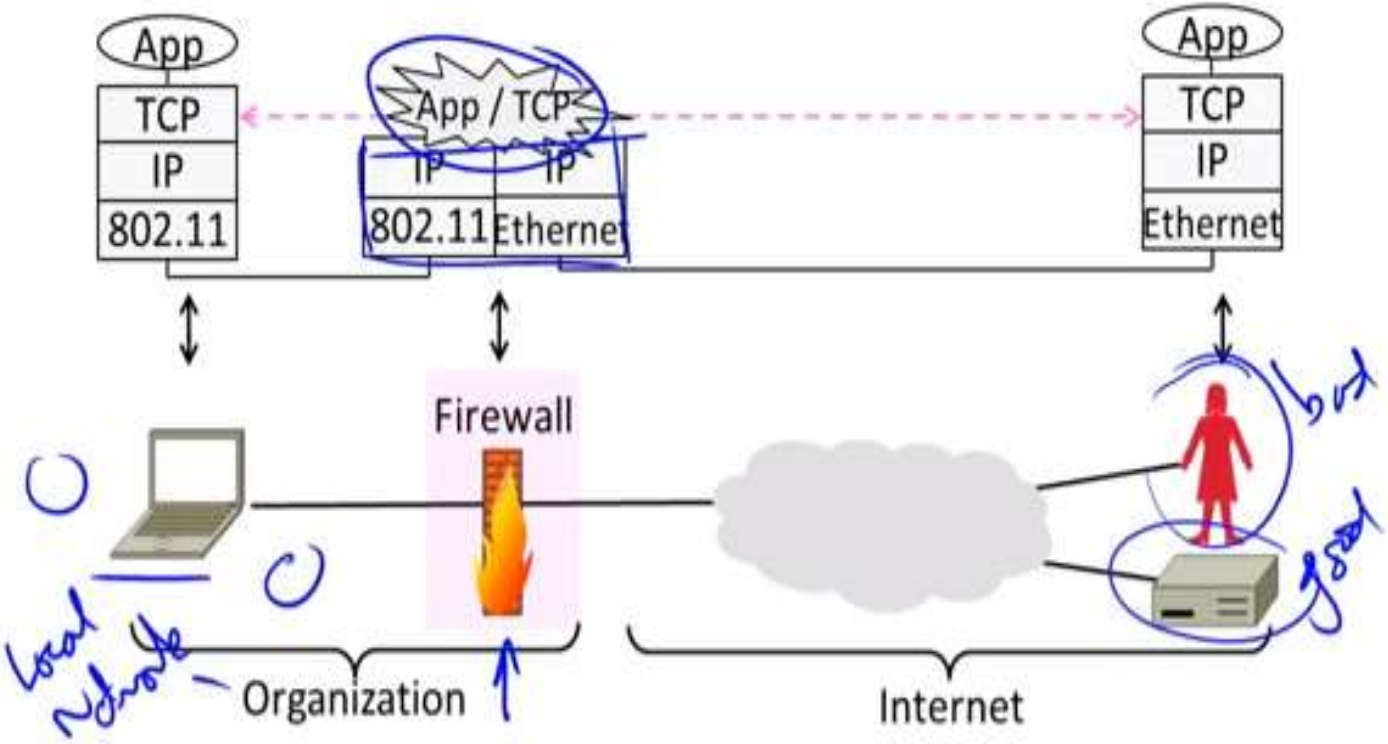


Recall Middleboxes

- Sit "inside the network" but perform "more than IP" processing on packets to add new functionality
  - NAT box, Firewall / Intrusion Detection System

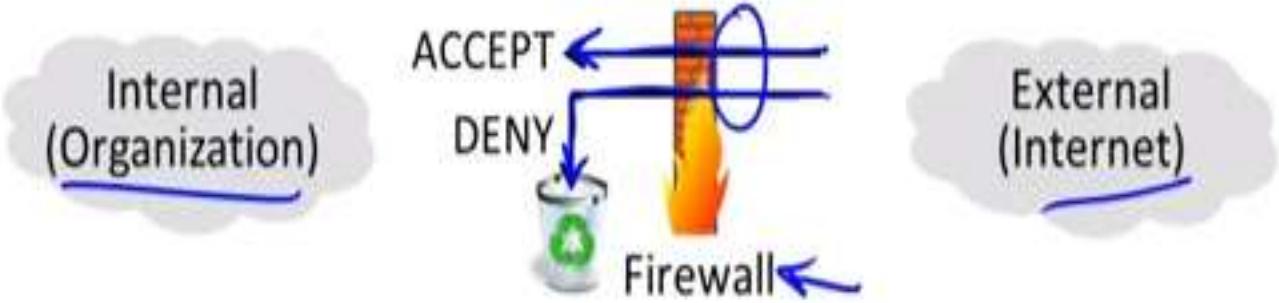


Firewall as Middlebox



Operation

- Firewall has two sides:
  - Internal (organization) and external (Internet)
- For each packet that tries to cross, decide whether to:
  - ACCEPT = pass unaltered; or DENY = discard silently
  - Decision is a local policy; firewall centralizes IT job



Design

- Key tension:
  - How to translate desired policies into packet filtering rules
- Policies are high-level statements
  - Relate to usage of apps, content
- Packet filtering is low-level
  - Limited viewpoint in the network, e.g., no app messages, encryption

Design (2)

- Stateless firewall
  - Simplest kind of firewall
  - Implements static packet filter rules
  - Typically using TCP/UDP ports
  - E.g., deny TCP port 23 (telnet)
  - Can allow/disallow many types of services and destinations

Design (3)

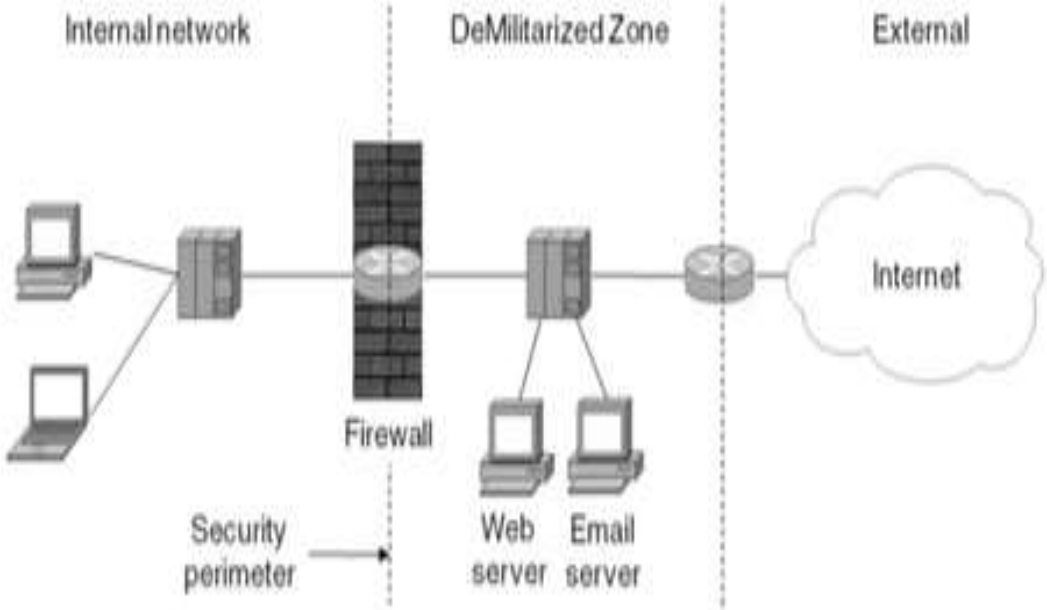
- Stateful firewall
  - A step up from stateless
  - Implements stateful packet filter rules that track packet exchanges
  - NAT example: accept incoming TCP packets after internal host connects

Design (4)

- Application layer firewall:
  - Another step up
  - Implements rules based on app usage and content
  - E.g., inspect content for viruses
  - Tries to look beyond packets by emulating higher layers, e.g., by reassembling app messages

Deployment

- Firewall is placed around internal/external boundary
  - Classic setup includes DMZ (DeMilitarized Zone) to put busy Internet hosts on the outside for better separation

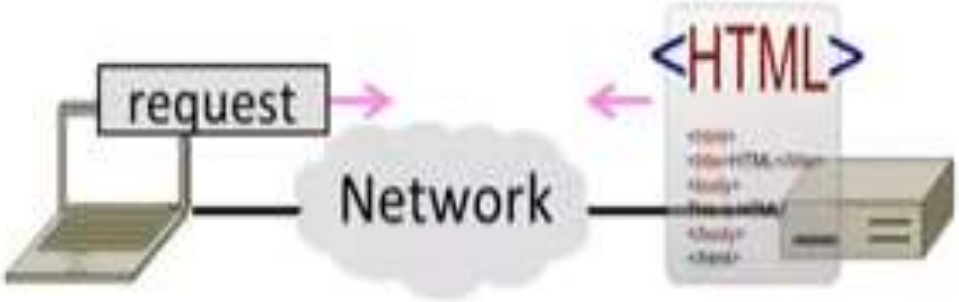


Deployment (2)

- Various device options:
  - Specialized network firewall
  - Firewall in boundary device, e.g., AP
  - Firewall as part of host, e.g., in OS
- Tradeoff:
  - Centralizing simplifies IT job
  - Distributing improves protection, visibility into apps, and performance

Topic

- HTTP, (HyperText Transfer Protocol)
  - Basis for fetching Web pages

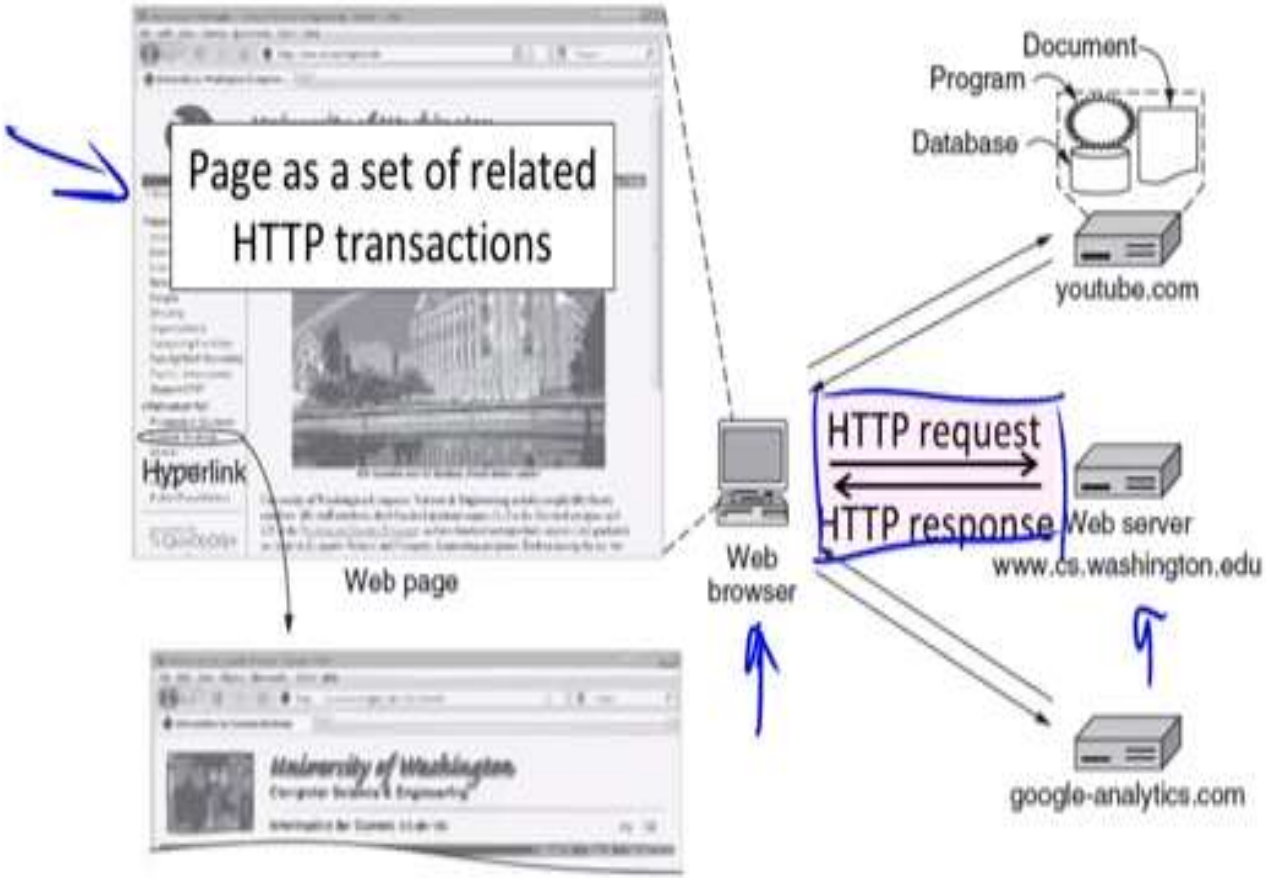


Sir Tim Berners-Lee (1955–)

- Inventor of the Web
  - Dominant Internet app since mid 90s
  - He now directs the W3C
- Developed Web at CERN in '89
  - Browser, server and first HTTP
  - Popularized via Mosaic ('93), Netscape
  - First WWW conference in '94 …

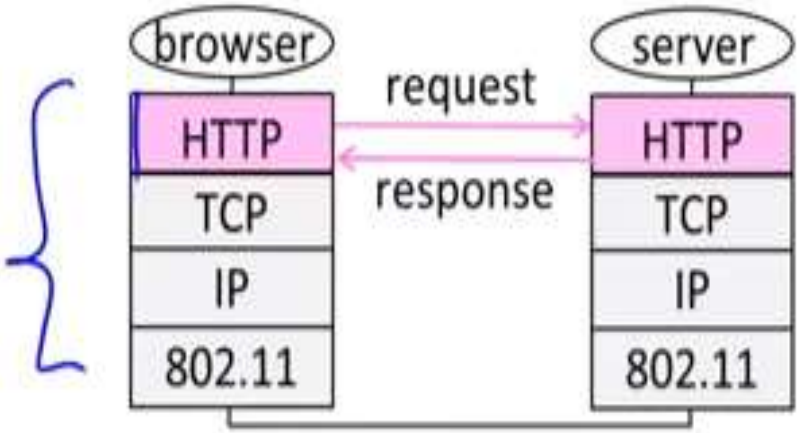
Source: By Paul Clarke, CC-BY-2.0, via Wikimedia Commons

Web Context


Page as a set of related HTTP transactions
Hyperlink
Web page
University of Washington
Computer Science & Engineering
Web browser
HTTP request
HTTP response
Document
Program
Database
youtube.com
Web server
www.cs.washington.edu
google-analytics.com

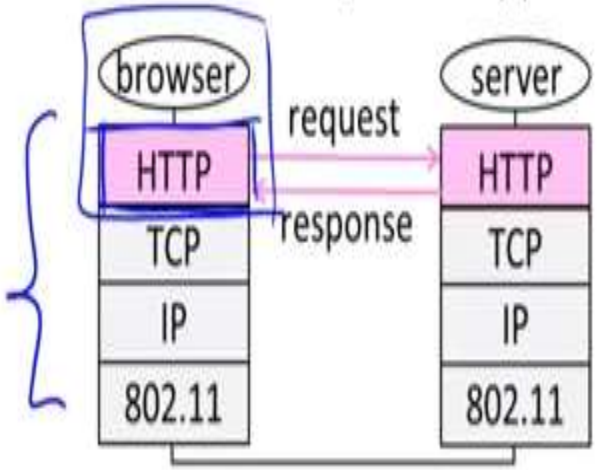
HTTP Context

- HTTP is a request/response protocol for fetching Web resources
  - Runs on TCP, typically port 80
  - Part of browser/server app



HTTP Context

- HTTP is a request/response protocol for fetching Web resources
  - Runs on TCP, typically port 80
  - Part of browser/server app



Fetching a Web page with HTTP

- Start with the page URL:

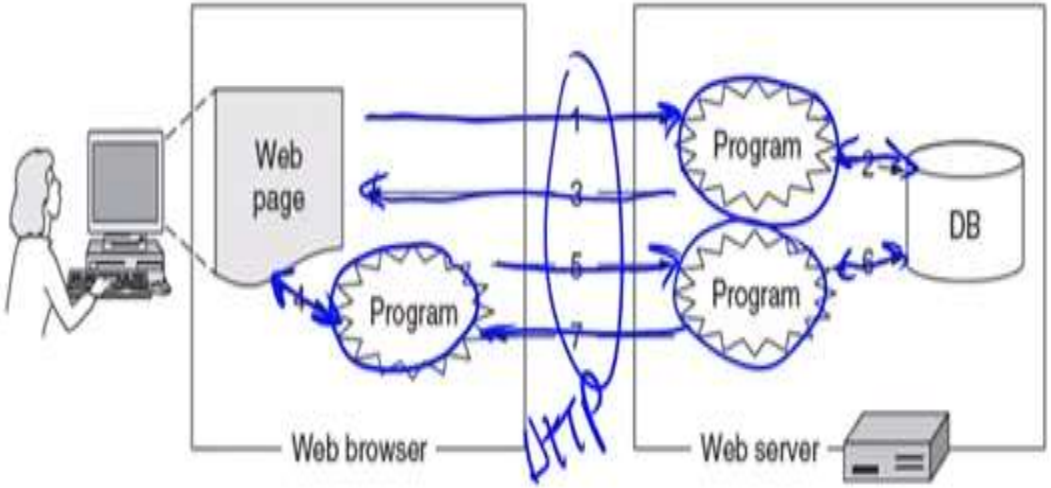
  http://en.wikipedia.org/wiki/Vegemite

  Protocol | Server | Page on server

- Steps:
  - Resolve the server to IP address (DNS)
  - Set up TCP connection to the server
  - Send HTTP request for the page
  - (Await HTTP response for the page)
  - ** Execute / fetch embedded resources / render
  - Clean up any idle TCP connections

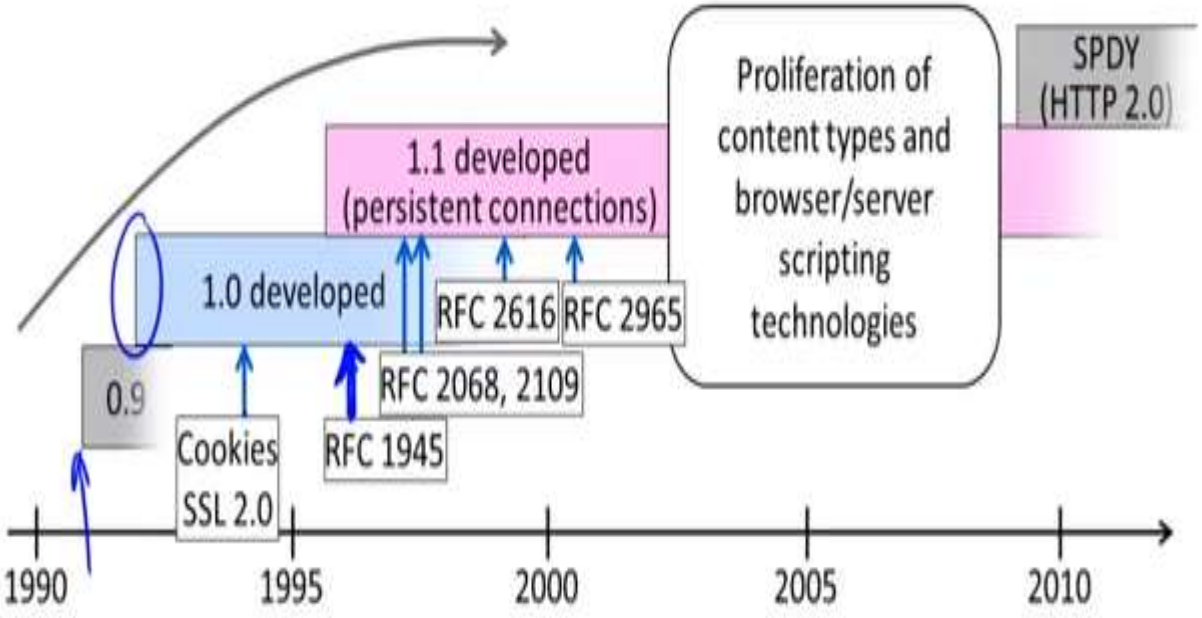
Static vs Dynamic Web pages

- Static web page is a file contents, e.g., image
- Dynamic web page is the result of program execution
  - Javascript on client, PHP on server, or both



Evolution of HTTP

- Consider security (SSL/TLS for HTTPS) later



HTTP Protocol

- Originally a simple protocol, with many options added over time
  - Text-based commands, headers
- Try it yourself:
  - As a "browser" fetching a URL
  - Run "telnet en.wikipedia.org 80"
  - Type "GET /wiki/Vegemite HTTP/1.0" to server followed by a blank line
  - Server will return HTTP response with the page contents (or other info)

HTTP Protocol (2)

- Commands used in the request

	Method	Description
Fetch page →	GET	Read a Web page
	HEAD	Read a Web page's header
Upload data →	POST	Append to a Web page
	PUT	Store a Web page
	DELETE	Remove the Web page
	TRACE	Echo the incoming request
	CONNECT	Connect through a proxy
	OPTIONS	Query options for a page

HTTP Protocol (3)

- Codes returned with the response

Code	Meaning	Examples
1xx	Information	100 = server agrees to handle client's request
2xx	Success	200 = request succeeded; 204 = no content present
3xx	Redirection	301 = page moved; 304 = cached page still valid
4xx	Client error	403 = forbidden page; 404 = page not found
5xx	Server error	500 = internal server error; 503 = try again later

Yes! → 2xx

HTTP Protocol (4)

- Many header fields specify capabilities and content
  - E.g., Content-Type: text/html, Cookie: lect=8-4-http

Function	Example Headers
Browser capabilities (client → server)	User-Agent, Accept, Accept-Charset, Accept-Encoding, Accept-Language
Caching related (mixed directions)	If-Modified-Since, If-None-Match, Date, Last-Modified, Expires, Cache-Control, ETag
Browser context (client → server)	Cookie, Referer, Authorization, Host
Content delivery (server → client)	Content-Encoding, Content-Length, Content-Type, Content-Language, Content-Range, Set-Cookie

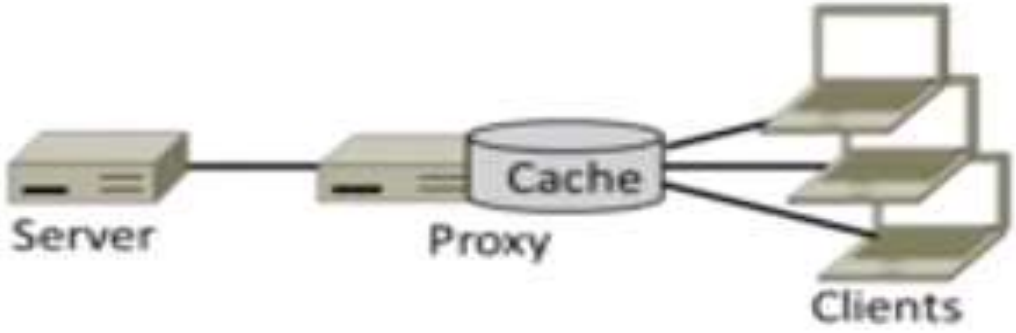
HTTP Protocol (4)

- Many header fields specify capabilities and content
  - E.g., Content-Type: text/html, Cookie: lect=8-4-http

Function	Example Headers
Browser capabilities (client → server)	User-Agent, Accept, Accept-Charset, Accept-Encoding, Accept-Language
Caching related (mixed directions)	If-Modified-Since, If-None-Match, Date, Last-Modified, Expires, Cache-Control, ETag
Browser context (client → server)	Cookie, Referer, Authorization, Host
Content delivery (server → client)	Content-Encoding, Content-Length, Content-Type, Content-Language, Content-Range, Set-Cookie

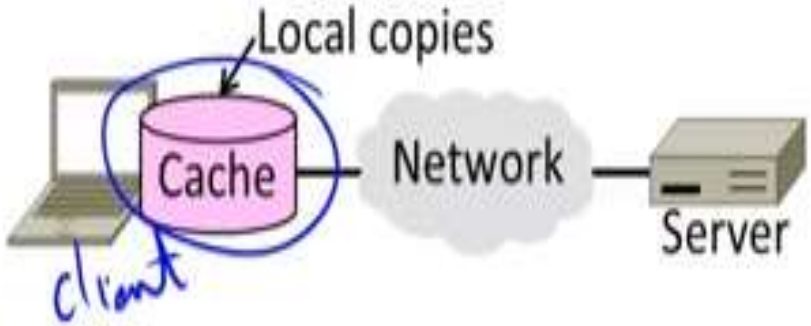
Topic

- HTTP caching and proxies
  - Enabling content reuse



Web Caching

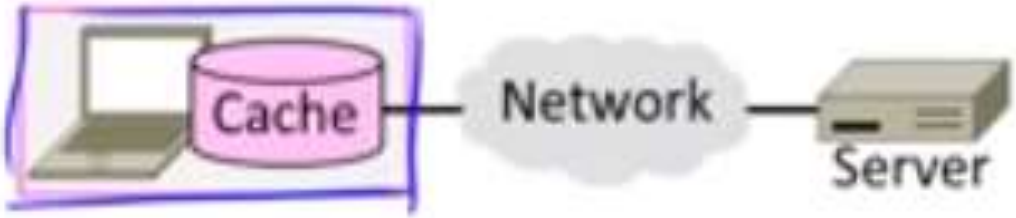
- Users often revisit web pages
  - Big win from reusing local copy!
  - This is caching



- Key question:
  - When is it OK to reuse local copy?

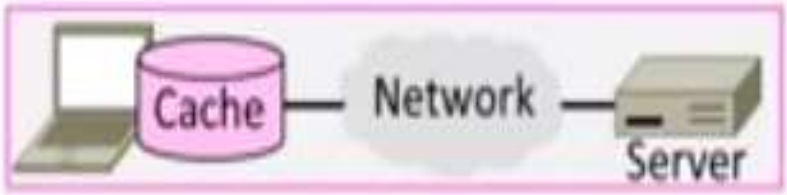
Web Caching (2)

- Locally determine copy is still valid
  - Based on expiry information such as "Expires" header from server
  - Or use a heuristic to guess (cacheable freshly valid, not modified recently)
  - Content is then available right away



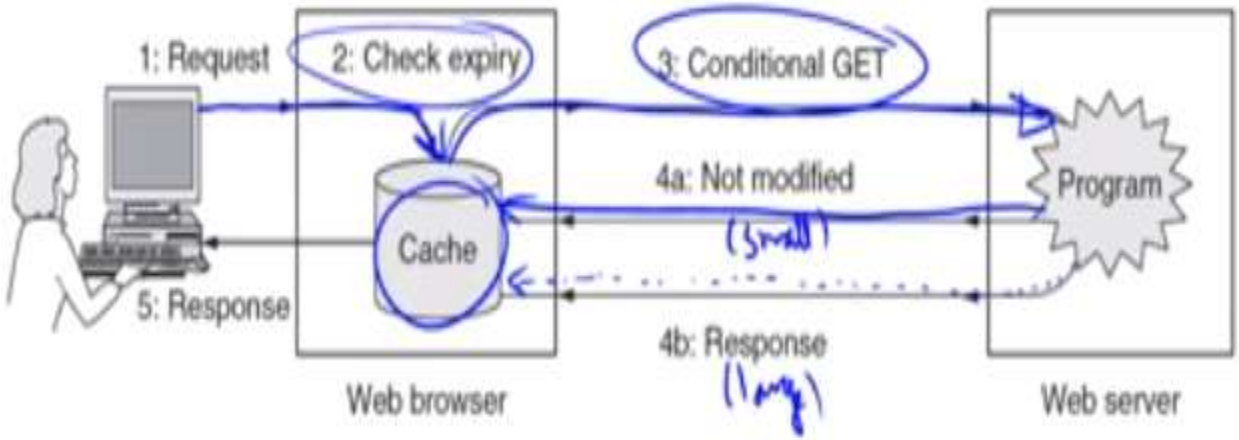
Web Caching (3)

- Revalidate copy with remote server
  - Based on timestamp of copy such as "Last-Modified" header from server
  - Or based on content of copy such as "Etag" header from server
  - Content is available after 1 RTT



Web Caching (4)

- Putting the pieces together:

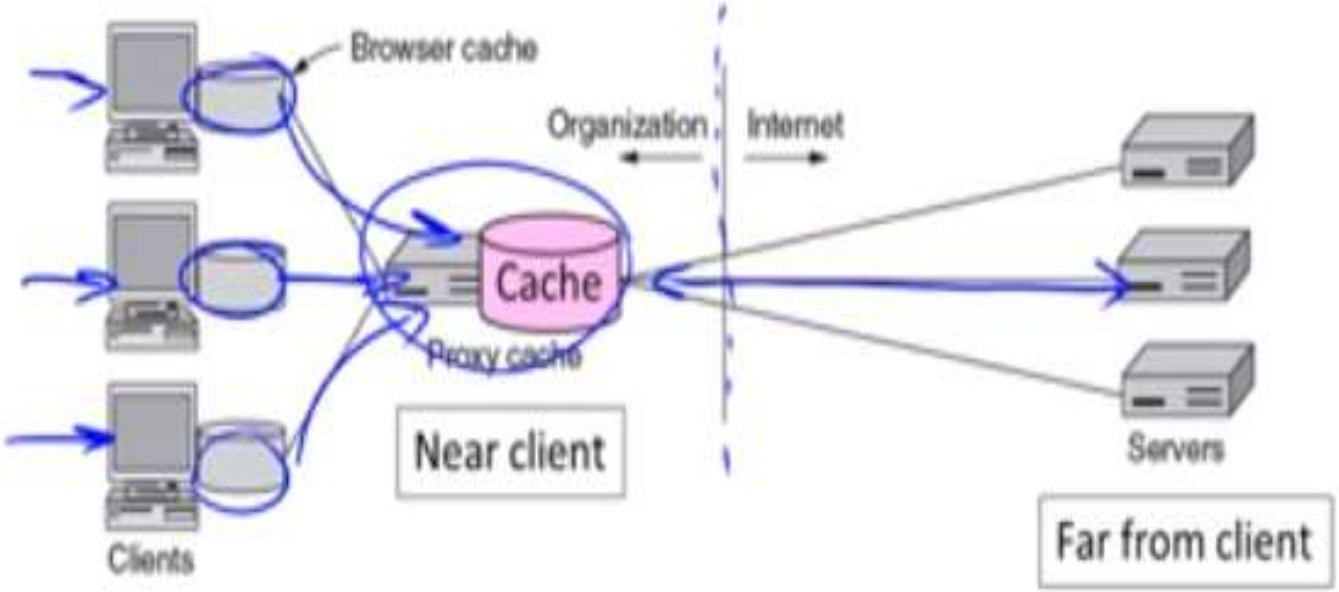


Web Proxies

- Place intermediary between pool of clients and external web servers
  - Benefits for clients include greater caching and security checking
  - Organizational access policies too!
- Proxy caching
  - Clients benefit from larger, shared cache
  - Benefits limited by secure / dynamic content, as well as "long tail"

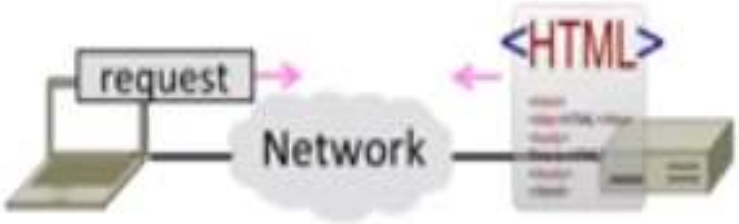
Web Proxies (2)

- Clients contact proxy; proxy contacts server



Topic

- Performance of HTTP
  - Parallel and persistent connections

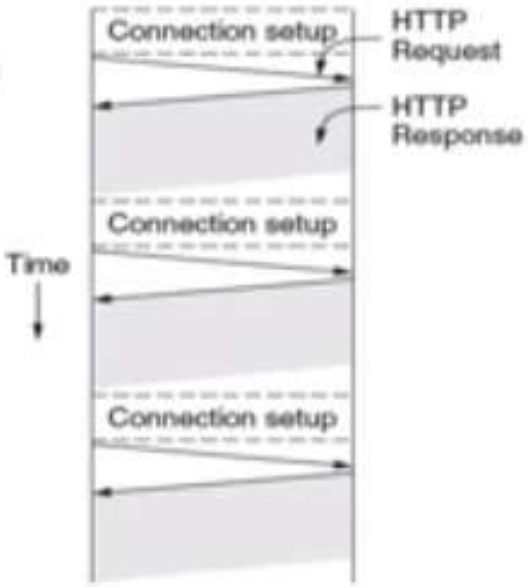


PLT (Page Load Time)

- PLT is the key measure of web performance
  - From click until user sees page
  - Small increases in PLT decrease sales
- PLT depends on many factors
  - Structure of page/content
  - HTTP (and TCP!) protocol
  - Network RTT and bandwidth

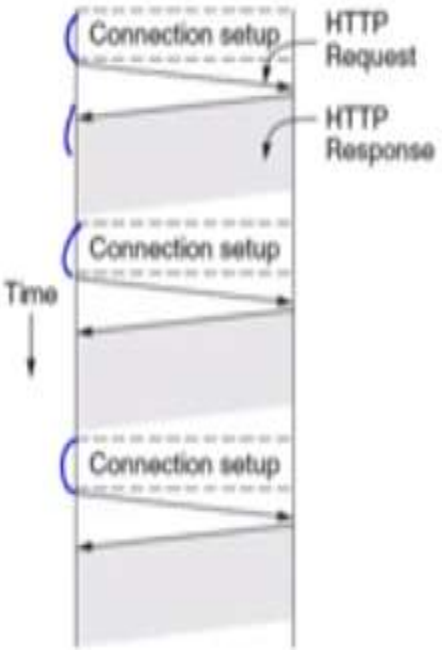
Early Performance (2)

- HTTP/1.0 used one TCP connection to fetch one web resource
  - Made HTTP very easy to build
  - But gave fairly poor PLT…



Early Performance (3)

- Many reasons why PLT is larger than necessary
  - Sequential request/responses, even when to different servers
  - Multiple TCP connection setups to the same server
  - Multiple TCP slow-start phases
- Network is not used effectively
  - Worse with many small resources / page



Ways to Decrease PLT

1. Reduce content size for transfer
   - Smaller images, gzip
2. Change HTTP to make better use of available bandwidth
3. Change HTTP to avoid repeated transfers of the same content
   - Caching, and proxies
4. Move content closer to client
   - CDNs [later]

This time

Next time

Later

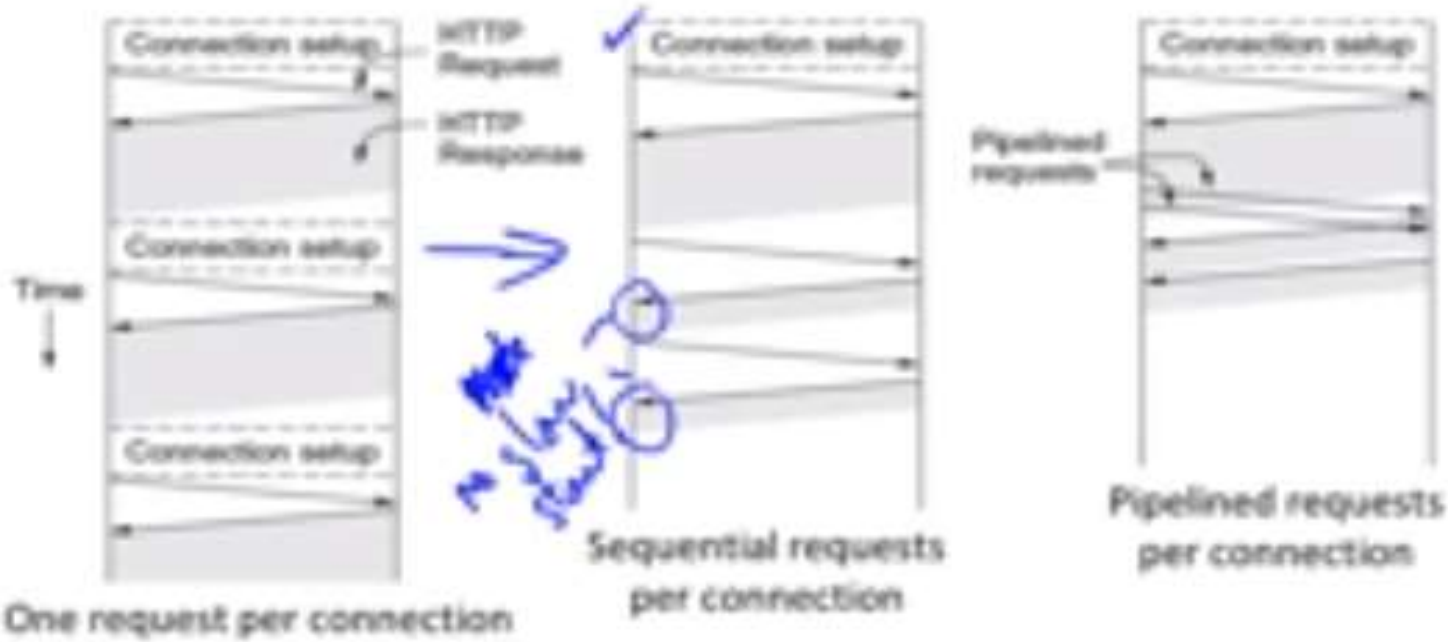
Parallel Connections

- One simple way to reduce PLT
  - Browser runs multiple (8, say) HTTP instances in parallel
  - Server is unchanged; already handled concurrent requests for many clients
- How does this help?
  - Single HTTP wasn't using network much ...
  - So parallel connections aren't slowed much
  - Pulls in completion time of last fetch

Persistent Connections

- Parallel connections compete with each other for network resources
  - 1 parallel client ≈ 8 sequential clients?
  - Exacerbates network bursts, and loss
- Persistent connection alternative
  - Make 1 TCP connection to 1 server
  - Use it for multiple HTTP requests

Persistent Connections (3)



Persistent Connections (4)

- Widely used as part of HTTP/1.1
  - Supports optional pipelining
  - PLT benefits depending on page structure, but easy on network
- Issues with persistent connections
  - How long to keep TCP connection?
  - Can it be slower? (Yes. But why?)